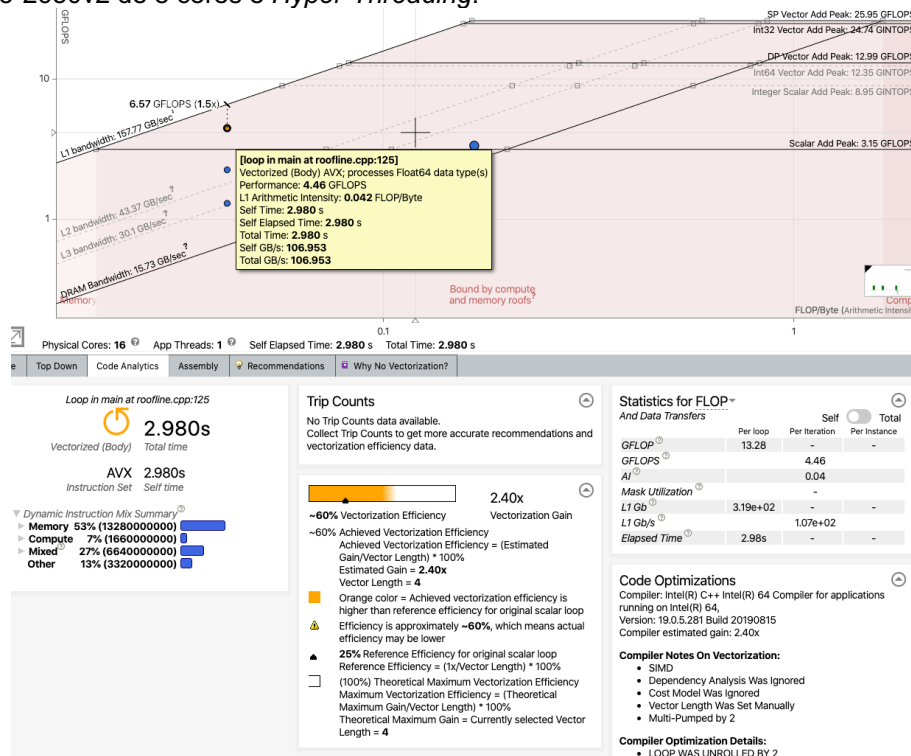




1. **Indique**, justificando, os passos envolvidos no processo de medição e otimização de código.
2. Um dos principais limitadores de desempenho atualmente é a longa latência no acesso à memória, quando comparado com a frequência do *clock* do CPU. **Analise e compare** a estratégia seguida pelos seguintes fabricantes de unidades de computação para ultrapassar esta limitação:
 - (i) Intel/AMD, nos seus processadores *multicore*;
 - (ii) Fujitsu, na arquitetura do seu processador *A64FX*;
 - (iii) NVidia, na arquitetura dos processadores Ampere.
3. Considere o seguinte *roofline* obtido para um algoritmo que opera sobre *arrays*. Quatro versões foram desenvolvidas: AoS sem vectorização; SoA sem vectorização; SoA com vectorização; SoA sem vectorização mas com uma implementação que faz mais cálculos para os mesmos dados. O servidor utilizado contém 2 Intel Xeon E5-2650v2 de 8 cores e *Hyper-Threading*.



- a) **Indique**, justificando, a qual ponto no gráfico *roofline* corresponde cada uma das quatro versões do algoritmo.
 - b) **Indique**, justificando, qual o *ceiling* que limita a performance do algoritmo mais próximo de 1 GFLOP.
 - c) Nem todos os *ceilings* do *roofline* são relevantes para este problema. **Indique**, justificando, quais os *ceilings* que deveriam ser removidos desta análise.
- 4.
- a) **Desenvolva** uma implementação eficiente em CUDA de um algoritmo que some todos os elementos de um array *in* com 4 vizinhos (2 à esquerda e 2 à direita) e guarde o resultado num array *out*.

```
__global__  
void sum (float *in, float *out, float vector_size) {  
    unsigned tid = threadIdx.x + blockIdx.x * blockDim.x;  
    // ...  
}
```

- b) **Implemente** o código acima em SYCL. Discuta as modificações necessárias para garantir processamento simultâneo em CPU e GPU.